

Module 12 – DevOps and CI/CD

1. What is DevOps?

Definition

DevOps is a software development methodology that combines:

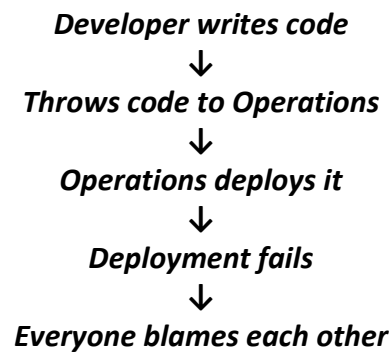
- **Dev (Development)** → Developers write code.
- **Ops (Operations)** → Operations team deploys, manages, and maintains applications.

Instead of working separately, they work **together throughout the software lifecycle**.

Simple Definition: DevOps is a culture and set of practices that integrates software development and IT operations to deliver applications faster, with better quality, and greater reliability.

Why was DevOps introduced?

Earlier, Development and Operations worked separately.

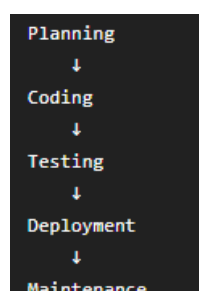


Problems:

- Slow deployment
- Communication gap
- Many bugs
- Manual work
- Delayed releases

DevOps solved these problems by encouraging collaboration and automation.

Traditional Software Development:



Each phase handled by different teams. Communication was poor.

DevOps Workflow:



This cycle repeats continuously.

Goals of DevOps:

DevOps aims to:

1. Faster Software Delivery: Develop software quickly, Instead of Release every 6 months. But Companies now release Several times a day.

Example:

Netflix updates frequently.

2. Better Collaboration: Developers and Operations work together. Instead of Developer vs Operations. It becomes Developer + Operations

3. Automation: Replace manual tasks with automation.

Examples:

- Automatic Testing
- Automatic Deployment
- Automatic Build

4. Better Quality: Frequent testing catches bugs early. Less chance of production failures.

5. Continuous Improvement: Every deployment provides feedback. Developers fix issues quickly.

6. Faster Bug Fixes: When bugs occur, Developers immediately fix and redeploy.

Benefits of DevOps:

Benefit	Explanation
Faster Delivery	Faster software releases
Better Collaboration	Teams work together
Reduced Errors	Automation reduces mistakes
Improved Quality	Continuous testing
Higher Customer Satisfaction	Quick feature delivery

Faster Recovery	Easier rollback after failures
Better Security	Security integrated throughout development (DevSecOps)
Scalability	Easier infrastructure management

DevOps Lifecycle

There are **8 stages**.

1. Planning: Requirements are collected.

Example: Customer wants Login, Registration, Payment. Tools used are Jira, Trello, Azure Boards.

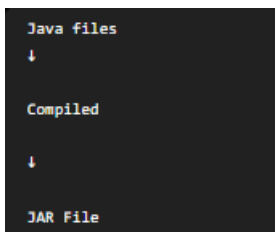
2. Development: Developers write source code.

Languages used are Java, Python, C# and JavaScript.

Version Control: Git

3. Build: Source code becomes executable software.

Example



Tools used are Maven, Gradle and Ant.

4. Continuous Integration (CI): Whenever code is pushed, Automatically Builds, Tests, Verify. No manual effort.

5. Testing: Automated testing.

Examples: Unit Testing, Integration Testing, UI Testing.

Tools used are Selenium, Junit and TestNG.

6. Deployment: Application moves to Testing Server, Staging Server and Production Server. Deployment can be automatic.

7. Monitoring: Application health monitored. It Checks CPU, Memory, Errors and User Requests. Tools used are Prometheus, Grafana and Nagios.

8. Feedback: Customer feedback collected, Improvements planned and Cycle repeats.

Key DevOps Practices:

1. Continuous Integration (CI): Developers merge code frequently. Every change is automatically Built and Tested. Detect bugs early.

Example

Developer A

adds login.

Developer B

adds payment.

Both upload code.

CI automatically checks how to Compile, Run Tests and Report Errors.

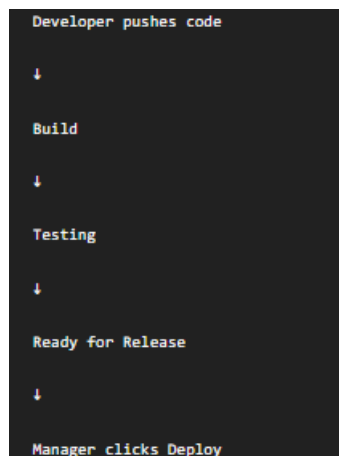
Advantages:

Detect bugs early, better code quality, Easy collaboration and Faster development.

2. Continuous Delivery (CD):

After successful testing, Software is always ready for deployment and Deployment requires only one approval.

Example:

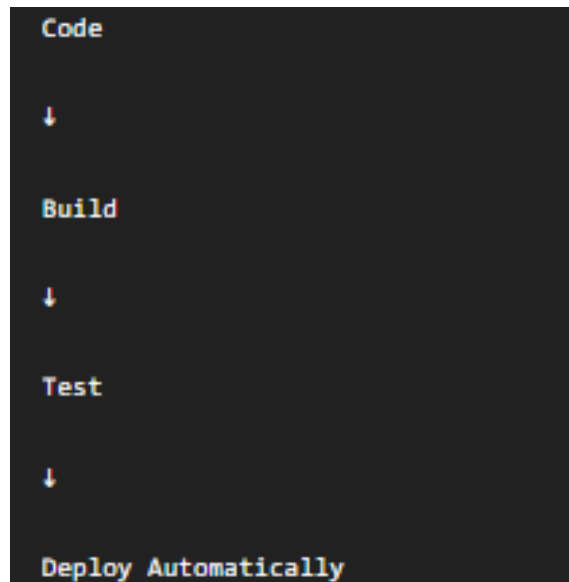


Advantages are Faster releases, less manual work and Reliable deployment.

3. Continuous Deployment:

Everything is automatic. No human approval.

Pipeline:



Used by Netflix, Amazon and Facebook.

Difference are Delivery, Needs approval, Deployment and No approval.

4. Infrastructure as Code (IaC):

Servers created using code. Instead of manually configuring servers, Write scripts.

Example tools are Terraform, Ansible and CloudFormation.

Advantages are Fast, Repeatable and Less human error.

5. Continuous Monitoring:

Monitor application continuously.

Examples are Server CPU becomes, 95% and Alert generated.

6. Version Control:

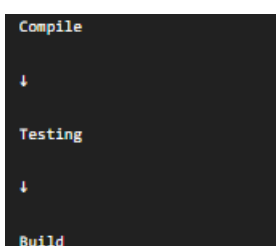
Stores source code history. Popular Tool and Git.

Benefits are Rollback, Collaboration and Track changes.

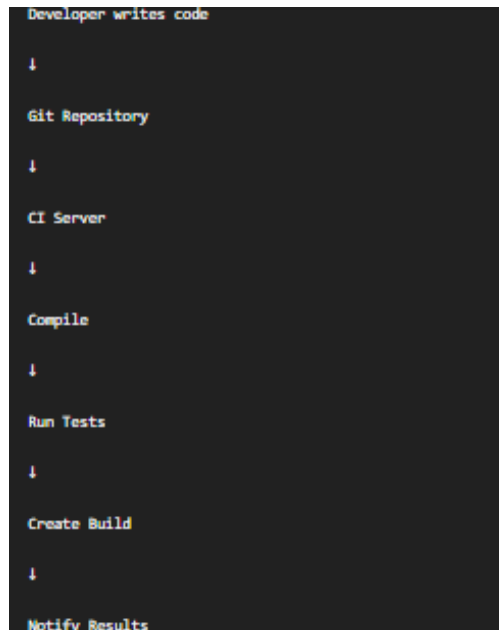
Continuous Integration (CI):

Definition: CI means developers merge code into a shared repository several times a day.

Each merge triggers automatically.



CI Process:

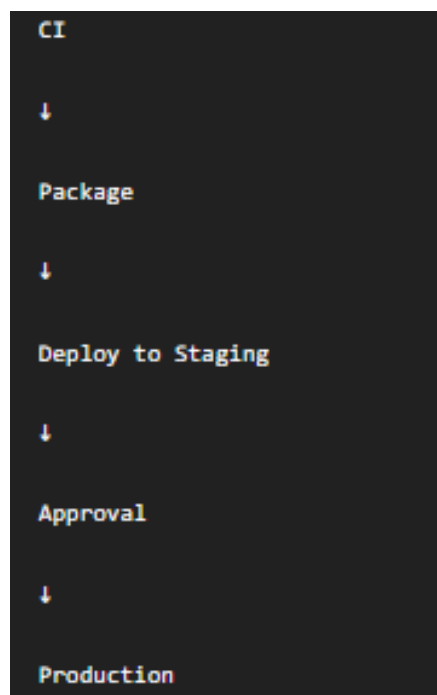


Advantages are Early bug detection, Better collaboration, Automatic testing and Stable software.

Continuous Delivery (CD):

Automatically prepares software for deployment. Final deployment requires manual approval.

Workflow:

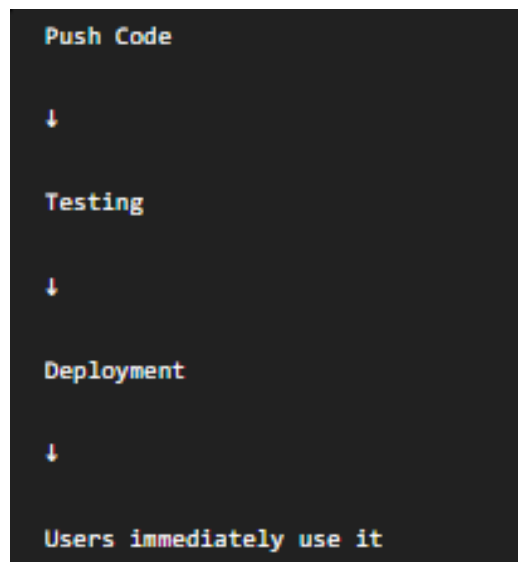


Advantages Faster releases, Less risk and Reliable deployment.

Continuous Deployment:

Every successful change is automatically deployed to production. No approval.

Example:



Advantages are Very fast and no manual work.

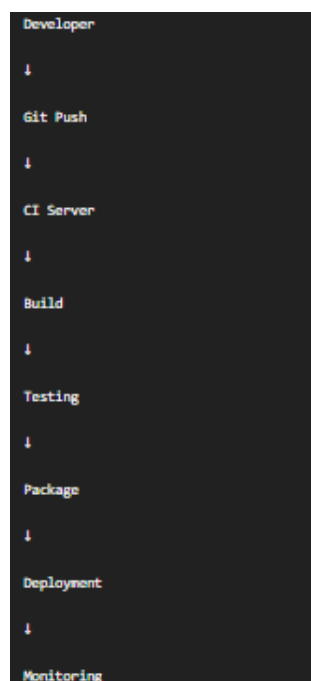
Disadvantages are a bug in tests may reach users if testing isn't strong.

Difference Between CI, Continuous Delivery, and Continuous Deployment

Feature	CI	Continuous Delivery	Continuous Deployment
Build Automation	Yes	Yes	Yes
Testing	Yes	Yes	Yes
Production Ready	No	Yes	Yes
Manual Approval	No	Yes	No
Automatic Production Deployment	No	No	Yes

CI/CD Pipeline:

Pipeline means automated steps.



Benefits of CI/CD:

- 1 Faster Releases:** Automatic pipeline.
- 2 Better Code Quality:** Testing after every commit.
- 3 Less Human Error:** Automation replaces manual work.
- 4 Easy Rollback:** Previous versions available.
- 5 Frequent Deployment:** Small updates and Less risk.
- 6 Improved Customer Satisfaction:** Features delivered faster.

Popular DevOps Tools:

- 1. Git:** Purpose, Version Control and Developer collaboration.

Features are Branching, Merging, History and Rollback.

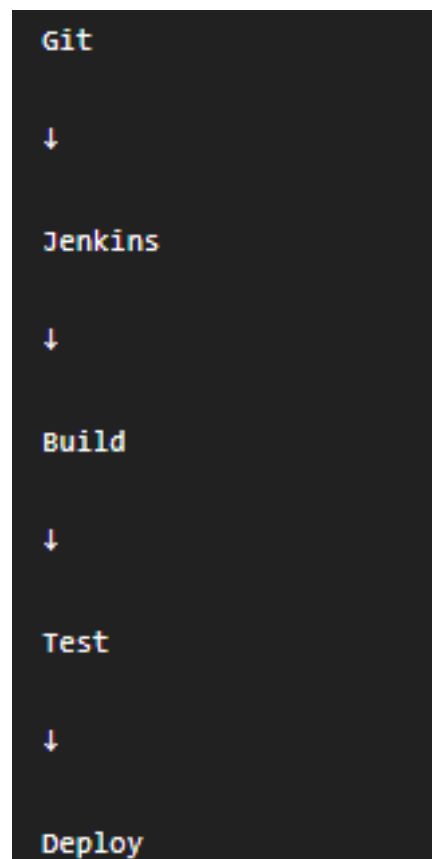
- 2. GitHub:** Cloud platform for Git repositories.

Features include Store code, Pull Requests, Issues and GitHub Actions.

- 3. Jenkins:** Most popular CI/CD automation tool.

Features include Build automation, Testing and Deployment.

Advantages are Open Source, Plugin support and Easy automation.



4. GitHub Actions :

CI/CD inside GitHub. No separate Jenkins server needed. Automatically executes.

Workflow file: `.github/workflows/main.yml`

Advantages are Easy setup, Integrated with GitHub and Free for many projects.

5. GitLab CI/CD:

Built into GitLab.

Uses: `.gitlab-ci.yml`

Pipeline

Build



Test



Deploy

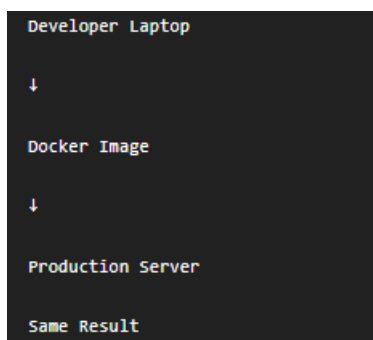
6. CircleCI :

Cloud CI/CD platform. It supports both the GitHub and Bitbucket. Some advantages are Fast builds, Easy configuration and Cloud based.

7. Docker:

The main Purpose is Containerization. Packages are Application, Libraries and Dependencies into one container. Works the same on every machine.

Example:



8. Kubernetes: Container orchestration. Manages many Docker containers.

Features are Auto Scaling, Load Balancing and Self-Healing.

9. Maven: Java build automation tool. Its functions are Compile, Test and Package.

10. Selenium: Automation testing, Used for Web application testing.

11. Ansible: Configuration Management and Automates server setup.

12. Terraform: Infrastructure as Code and Creates cloud infrastructure using code.

DevOps Toolchain:

Planning

Jira



Code

Git



Repository

GitHub



Build

Maven



CI/CD

Jenkins



Container

Docker



Orchestration

Kubernetes



Monitoring

Prometheus + Grafana

- **DevOps:** Culture that combines **Development + Operations** to deliver software faster and more reliably.
- **Goals:** Faster delivery, collaboration, automation, quality, continuous improvement.
- **Lifecycle:** Planning → Development → Build → CI → Testing → Deployment → Monitoring → Feedback.

- **CI (Continuous Integration):** Automatically builds and tests code whenever developers merge changes.
- **Continuous Delivery:** Software is always deployment-ready, but production deployment requires **manual approval**.
- **Continuous Deployment:** Every successful change is **automatically deployed** to production.
- **Popular Tools:** Git, GitHub, Jenkins, GitHub Actions, GitLab CI/CD, CircleCI, Maven, Docker, Kubernetes, Selenium, Ansible, Terraform, Prometheus, Grafana.